

SmartBranch 360

Individual Summary Document — Member 3: Python Tool, Documentation & Demo

Field	Details
Student Name	Akhil Gwari
Roll Number	2410030441
College	IILM University, Greater Noida
Technology / Branch	B.Tech — Computer Science Engineering (CSE)
Project Title	SmartBranch 360 — Secure Branch-Office Network
Platform	Cisco Packet Tracer
Group Size	3 Members
My Contribution Area	Member 3 — Python Tool, Documentation & Demo (smartbranch_checker.py, requirements.yaml, design document compilation, fault-card documentation, demo recording)

Smart Branch Network

Verification & Documentation Report

Prepared by: Member 3 — Python Tool, Documentation & Demo

1. Project Overview

This report documents the Smart Branch Network project built in Cisco Packet Tracer, and presents the results of the automated configuration checker (smartbranch_checker.py) developed to verify the network build against the agreed design requirements.

The network consists of four VLANs — Employee, Guest, Server, and Management — connected through a Router-on-a-Stick topology, with DHCP services, inter-VLAN security restrictions, NAT for internet access, and SSH management access restricted to the Management VLAN.

2. Network Design Summary

2.1 VLAN & IP Addressing Plan

VLAN ID	Name	Network	Gateway
10	EMPLOYEE	10.10.10.0/24	10.10.10.1
20	GUEST	10.10.20.0/24	10.10.20.1
30	SERVER	10.10.30.0/24	10.10.30.1
99	MANAGEMENT	10.10.99.0/24	10.10.99.1

2.2 Trunking

- Trunk interface: Fa0/1 (switch-to-switch and switch-to-router links)
- Allowed VLANs on trunk: 10, 20, 30, 99
- Encapsulation: 802.1Q

2.3 Routing

- Router-on-a-Stick using sub-interfaces on GigabitEthernet0/0 (one sub-interface per VLAN)
- Each sub-interface acts as the default gateway for its VLAN

2.4 DHCP

- Four DHCP pools configured on the router — one per VLAN — each handing out addresses with the correct default gateway
- First 20 addresses in each subnet excluded for static/reserved use

2.5 Security Notes

- Guest VLAN is blocked from reaching the Server VLAN (extended ACL, GUEST-RESTRICT)
- Guest VLAN is blocked from reaching the Management VLAN
- Guest VLAN retains internet access via NAT
- SSH (version 2) is enabled for remote management, restricted to the Management VLAN only via access-class SSH-MANAGEMENT

2.6 NAT

- PAT (NAT overload) configured on the router's internet-facing interface (GigabitEthernet0/1), allowing all internal VLANs to reach the internet using a single public-facing address

3. Python Verification Tool — smartbranch_checker.py

A Python script was written to automatically verify the live device configuration against the requirements defined in requirements.yaml, without requiring any new network design work. The script reads exported "show running-config" and "show vlan/trunk" output from the router and switch, and checks each requirement using pattern matching.

3.1 What the script checks

- VLAN Configuration — confirms all 4 VLANs exist and are active on the switch
- Trunk Configuration — confirms all 4 VLANs are permitted on the Fa0/1 trunk
- Gateway Configuration — confirms each router sub-interface has the correct IP address
- DHCP Pools — confirms each pool has the correct network and default-router values
- Security Rules — confirms the Guest→Server and Guest→Management deny rules are present
- NAT — confirms the PAT overload rule is configured on the correct interface
- SSH — confirms SSH version 2 is enabled and management access is restricted to the Management VLAN

3.2 Inputs

- router_output.txt — exported running-config from R1
- sw1_output.txt — exported VLAN and trunk output from SW1
- requirements.yaml — the agreed design values used as the baseline

4. Verification Results

The checker was executed against the final exported configuration files. All 21 checks passed, confirming the build matches the requirements document.

Test Area	Result	Checks	Passed
VLAN Configuration	PASS	4 VLANs (10/20/30/99)	4 / 4
Trunk Configuration	PASS	4 VLANs allowed on Fa0/1	4 / 4
Router Sub-interfaces	PASS	4 gateway IPs	4 / 4
DHCP Pools	PASS	4 pools	4 / 4
Security (Guest ACL)	PASS	2 deny rules	2 / 2
NAT Overload	PASS	1 rule	1 / 1
SSH Access Control	PASS	2 checks	2 / 2

Final Result: 21 / 21 checks passed — STATUS: ALL CHECKS PASSED

5. Conclusion

The Smart Branch Network build satisfies all VLAN, trunking, routing, DHCP, security, NAT, and SSH requirements defined at the start of the project. The automated checker confirms the configuration is consistent and complete.

Appendix A: smartbranch_checker.py — Full Source Code

The complete source code of the Python verification tool is included below for reference, including the main() function and script entry point.

```
import re

ROUTER_FILE = "router_output.txt"
SWITCH_FILE = "sw1_output.txt"

def read_file(filename):
    try:
        with open(filename, "r") as file:
            return file.read()
    except FileNotFoundError:
        print(f"[ERROR] {filename} not found!")
        return ""

def check_vlans(config):
    print("\n--- VLAN CHECK ---")

    expected = {
        "10": "EMPLOYEE",
        "20": "GUEST",
        "30": "SERVER",
        "99": "MANAGEMENT"
    }

    passed = 0

    for vlan_id, vlan_name in expected.items():
        pattern = rf"{vlan_id}\s+{vlan_name}\s+active"

        if re.search(pattern, config, re.IGNORECASE):
            print(f"[PASS] VLAN {vlan_id} - {vlan_name}")
            passed += 1
        else:
```

```

        print(f"[FAIL] VLAN {vlan_id} - {vlan_name}")

    return passed

def check_trunk(config):
    print("\n--- TRUNK CHECK ---")

    required_vlans = ["10", "20", "30", "99"]

    passed = 0

    for vlan in required_vlans:
        if re.search(
            rf"Fa0/1.*?{vlan}",
            config,
            re.IGNORECASE | re.DOTALL
        ):
            print(f"[PASS] VLAN {vlan} allowed on trunk")
            passed += 1
        else:
            print(f"[FAIL] VLAN {vlan} missing from trunk")

    return passed

def check_gateway(config):
    print("\n--- GATEWAY CHECK ---")

    gateways = {
        "10": "10.10.10.1",
        "20": "10.10.20.1",
        "30": "10.10.30.1",
        "99": "10.10.99.1"
    }

    passed = 0

    for vlan, gateway in gateways.items():
        pattern = (
            rf"interface GigabitEthernet0/0\.{vlan}"
            rf".*?ip address {re.escape(gateway)} "
            rf"255\.255\.255\.0"
        )

        if re.search(pattern, config, re.IGNORECASE | re.DOTALL):
            print(f"[PASS] VLAN {vlan} gateway {gateway}")
            passed += 1
        else:
            print(f"[FAIL] VLAN {vlan} gateway {gateway}")

    return passed

def check_dhcp(config):
    print("\n--- DHCP CHECK ---")

    pools = {

```

```

        "EMPLOYEE": ("10.10.10.0", "10.10.10.1"),
        "GUEST": ("10.10.20.0", "10.10.20.1"),
        "SERVER": ("10.10.30.0", "10.10.30.1"),
        "MANAGEMENT": ("10.10.99.0", "10.10.99.1")
    }

    passed = 0

    for pool, (network, gateway) in pools.items():

        pattern = (
            rf"ip dhcp pool {pool}"
            rf".*?network {re.escape(network)} 255\.255\.255\.0"
            rf".*?default-router {re.escape(gateway)}"
        )

        if re.search(pattern, config, re.IGNORECASE | re.DOTALL):
            print(f"[PASS] DHCP {pool}")
            passed += 1
        else:
            print(f"[FAIL] DHCP {pool}")

    return passed

def check_security(config):
    print("\n--- SECURITY CHECK ---")

    passed = 0

    guest_server = (
        "deny ip 10.10.20.0 0.0.0.255 "
        "10.10.30.0 0.0.0.255"
    )

    guest_management = (
        "deny ip 10.10.20.0 0.0.0.255 "
        "10.10.99.0 0.0.0.255"
    )

    if guest_server in config:
        print("[PASS] Guest -> Server blocked")
        passed += 1
    else:
        print("[FAIL] Guest -> Server rule missing")

    if guest_management in config:
        print("[PASS] Guest -> Management blocked")
        passed += 1
    else:
        print("[FAIL] Guest -> Management rule missing")

    return passed

def check_nat(config):
    print("\n--- NAT CHECK ---")

    rule = (
        "ip nat inside source list 1 "

```

```

        "interface GigabitEthernet0/1 overload"
    )

    if rule in config:
        print("[PASS] NAT overload configured")
        return 1

    print("[FAIL] NAT overload missing")
    return 0

def check_ssh(config):
    print("\n--- SSH CHECK ---")

    passed = 0

    if "ip ssh version 2" in config:
        print("[PASS] SSH version 2 enabled")
        passed += 1
    else:
        print("[FAIL] SSH version 2 missing")

    if "access-class SSH-MANAGEMENT in" in config:
        print("[PASS] SSH restricted to Management VLAN")
        passed += 1
    else:
        print("[FAIL] SSH management restriction missing")

    return passed

def main():

    print("=" * 55)
    print("    SMART BRANCH NETWORK ASSURANCE CHECKER")
    print("=" * 55)

    router = read_file(ROUTER_FILE)
    switch = read_file(SWITCH_FILE)

    if not router or not switch:
        return

    total = 0
    passed = 0

    # Switch checks
    result = check_vlans(switch)
    passed += result
    total += 4

    result = check_trunk(switch)
    passed += result
    total += 4

    # Router checks
    result = check_gateway(router)
    passed += result
    total += 4

```

```

    result = check_dhcp(router)
    passed += result
    total += 4

    result = check_security(router)
    passed += result
    total += 2

    result = check_nat(router)
    passed += result
    total += 1

    result = check_ssh(router)
    passed += result
    total += 2

    print("\n" + "=" * 55)
    print(f"RESULT: {passed}/{total} checks passed")
    print("=" * 55)

    if passed == total:
        print("STATUS: ALL CHECKS PASSED")
    else:
        print("STATUS: SOME CHECKS FAILED")

    print("=" * 55)

if __name__ == "__main__":
    main()

```

Appendix B: Program Output

The exact console output produced by running `smartbranch_checker.py` against the final `router_output.txt` and `sw1_output.txt` files is shown below.

```

=====
SMART BRANCH NETWORK ASSURANCE CHECKER
=====

--- VLAN CHECK ---
[PASS] VLAN 10 - EMPLOYEE
[PASS] VLAN 20 - GUEST
[PASS] VLAN 30 - SERVER
[PASS] VLAN 99 - MANAGEMENT

--- TRUNK CHECK ---
[PASS] VLAN 10 allowed on trunk
[PASS] VLAN 20 allowed on trunk
[PASS] VLAN 30 allowed on trunk
[PASS] VLAN 99 allowed on trunk

--- GATEWAY CHECK ---
[PASS] VLAN 10 gateway 10.10.10.1
[PASS] VLAN 20 gateway 10.10.20.1
[PASS] VLAN 30 gateway 10.10.30.1
[PASS] VLAN 99 gateway 10.10.99.1

```

--- DHCP CHECK ---

[PASS] DHCP EMPLOYEE

[PASS] DHCP GUEST

[PASS] DHCP SERVER

[PASS] DHCP MANAGEMENT

--- SECURITY CHECK ---

[PASS] Guest -> Server blocked

[PASS] Guest -> Management blocked

--- NAT CHECK ---

[PASS] NAT overload configured

--- SSH CHECK ---

[PASS] SSH version 2 enabled

[PASS] SSH restricted to Management VLAN

=====
RESULT: 21/21 checks passed

=====
STATUS: ALL CHECKS PASSED
=====